

Understanding the Complete Apache mod_auth_openidc Reverse Proxy Configuration

This project implements a centralized authentication architecture using:

- Apache HTTP Server
- mod_auth_openidc
- Microsoft Entra ID
- Azure Web App Service

The objective of this architecture is to protect backend applications without embedding authentication libraries such as MSAL/MSL directly into application code.

Instead of application-level authentication, authentication is moved into the infrastructure layer using Apache as a reverse proxy.

Final Architecture

```
graph TD; A[User Browser] --> B[Apache VM (mod_auth_openidc)]; B --> C[Microsoft Entra ID]; C --> D[Azure Web App Service];
```

User Browser
↓
Apache VM (mod_auth_openidc)
↓
Microsoft Entra ID
↓
Azure Web App Service



Authentication Flow

-
1. User accesses Apache VM
 2. Apache checks authentication session
 3. If unauthenticated → redirect to Microsoft login
 4. User authenticates using Entra ID
 5. Apache receives authorization code
 6. Apache exchanges code for token
 7. Apache validates token
 8. Apache creates session
 9. Apache forwards request to Azure Web App
 10. Web App returns response



Complete Configuration File

File:

/etc/apache2/sites-available/default-ssl.conf



Configuration:

⌞ apache



<VirtualHost *:443>

ServerName 40.67.138.103

ServerAdmin webmaster@localhost

SSLEngine on

SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem

SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key

OIDCProviderMetadataURL https://login.microsoftonline.com/d52cef87-e3e8-4faa-9723-6d961c7

OIDCClientID a667b207-05b0-46eb-af1a-ada9e41b8847

OIDCClientSecret YOUR_SECRET

OIDCRedirectURI https://40.67.138.103/redirect_uri

OIDCCryptoPassphrase randomSecret123

OIDCRemoteUserClaim sub

OIDCPassClaimsAs environment

OIDCScope "openid profile email"

SSLProxyEngine On

SSLProxyVerify none

SSLProxyCheckPeerCN off

SSLProxyCheckPeerName off

SSLProxyCheckPeerExpire off

ProxyPreserveHost Off

RequestHeader set X-Forwarded-Proto "https"

ProxyPass / https://webodic-d9fjhcdcd5b9bcfc.westus3-01.azurewebsites.net/

ProxyPassReverse / https://webodic-d9fjhcdcd5b9bcfc.westus3-01.azurewebsites.net/

<Location />

AuthType openid-connect

Require valid-user

</Location>

<Location /redirect_uri>

AuthType openid-connect

Require valid-user

</Location>

```
</VirtualHost>
```

Understanding `<VirtualHost *:443>`

```
</> apache
```



```
<VirtualHost *:443>
```

This tells Apache:

- listen on port 443
- handle HTTPS traffic
- apply all configuration inside this block

Why Port 443?

Port 443 is the standard HTTPS port.

HTTPS provides:

- encryption
- secure communication
- secure token exchange
- protection against packet sniffing

OpenID Connect requires HTTPS because:

- tokens are sensitive
- authentication cookies must be secure
- browser security policies require encrypted sessions

What is SSL/TLS?

SSL/TLS encrypts communication between:

- browser
- Apache server

Without SSL:

- passwords can be intercepted
- tokens can be stolen

- sessions can be hijacked

HTTPS uses:

- SSL certificates
- TLS encryption

Understanding SSLEngine on

```
</> apache  
  
SSLEngine on
```



This enables HTTPS support inside Apache.

Without this:

- Apache only serves HTTP
- OpenID Connect authentication becomes insecure

Understanding SSL Certificates

```
</> apache  
  
SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem  
SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key
```



These files contain:

- public certificate
- private encryption key

Purpose:

- encrypt traffic
- establish HTTPS session
- enable secure browser communication

Why Browser Shows “Not Secure”

Because:

- self-signed certificate used

- certificate does not match IP address

This is acceptable for testing.

Production systems use:

- trusted CA certificates
- Let's Encrypt
- enterprise SSL certificates

Understanding `OIDCProviderMetadataURL`

```
</> apache
```



```
OIDCProviderMetadataURL https://login.microsoftonline.com/TENANT_ID/v2.0/.well-known/openid-c
```

This is one of the MOST important directives.

Apache automatically downloads:

- authorization endpoint
- token endpoint
- signing keys
- issuer information

This allows Apache to communicate with Microsoft Entra ID.

Understanding `OIDCClientID`

```
</> apache
```



```
OIDCClientID a667b207-05b0-46eb-af1a-ada9e41b8847
```

This uniquely identifies Apache as an application registered in Microsoft Entra ID.

Think of it as:

- application username
 - identity of Apache server
-

Understanding `OIDCClientSecret`

```
</> apache
```



```
OIDCClientSecret YOUR_SECRET
```

This is the password for Apache.

Used during:

- token exchange
- secure authentication
- authorization code validation

Must always remain secret.

Understanding `OIDCRedirectURI`

```
</> apache
```



```
OIDCRedirectURI https://40.67.138.103/redirect_uri
```

After Microsoft login:

- user returns to this endpoint

Apache receives:

- authorization code
- authentication response

Then Apache exchanges the code for tokens.

Understanding `OIDCCryptoPassphrase`

```
</> apache
```



```
OIDCCryptoPassphrase randomSecret123
```

Apache uses this to:

- encrypt sessions
- protect cookies
- secure token storage

Acts as internal encryption password.

Understanding `OIDCRemoteUserClaim`

```
</> apache
```



```
OIDCRemoteUserClaim sub
```

Defines:

which token claim becomes the authenticated user.

The `sub` claim:

- uniquely identifies users
 - always exists
 - stable across sessions
-

What are Claims?

Claims are user attributes inside tokens.

Example:

```
</> JSON
```



```
{  
  "sub": "123456",  
  "email": "user@example.com"  
}
```

Claims may contain:

- email
 - username
 - groups
 - tenant ID
 - object ID
-

Understanding `OIDCPassClaimsAs`

```
</> apache
```



```
OIDCPassClaimsAs environment
```

This passes claims into backend application environment variables.

Backend apps can read:

- user identity
- email
- group membership

Understanding OIDCScope

```
</> apache
```



```
OIDCScope "openid profile email"
```

Scopes define what information Apache requests.

Scope Explanation

Scope	Purpose
openid	Enables OpenID Connect authentication
profile	Requests profile information
email	Requests email claim

Why openid is Mandatory

Without:

```
</> apache
```



```
openid
```

authentication fails because:

- request becomes OAuth
- not OpenID Connect

Understanding Reverse Proxy

Apache acts as a reverse proxy.

Instead of users accessing:

Azure Web App directly



users access:

Apache VM



Apache then forwards requests internally.

Understanding SSLProxyEngine On

```
<> apache
```



```
SSLProxyEngine On
```

Allows Apache to establish HTTPS connection to backend Azure Web App.

Without this:

- Apache cannot proxy HTTPS traffic

Understanding SSL Proxy Settings

```
<> apache
```



```
SSLProxyVerify none
```

```
SSLProxyCheckPeerCN off
```

```
SSLProxyCheckPeerName off
```

These disable strict SSL verification.

Used because:

- Azure App Service TLS handshake caused issues
- testing environment

Production systems should use proper certificate validation.

Understanding ProxyPass

```
</> apache
```



```
ProxyPass / https://webodic-d9fjhcdcd5b9bcfc.westus3-01.azurewebsites.net/
```

This forwards incoming requests to backend Azure Web App.

Apache becomes:

- gateway
 - traffic controller
-

Understanding ProxyPassReverse

```
</> apache
```



```
ProxyPassReverse / https://webodic-d9fjhcdcd5b9bcfc.westus3-01.azurewebsites.net/
```

Rewrites backend responses correctly.

Prevents:

- redirect issues
 - URL mismatches
-

Understanding ProxyPreserveHost Off

```
</> apache
```



```
ProxyPreserveHost Off
```

Prevents Apache from forwarding original host header.

This fixed:

- Azure SSL handshake issue
-

Understanding `<Location />`

```
</> apache  
  
<Location />
```



Protects entire website.

All requests require:

- authentication
 - valid session
-

Understanding `AuthType openid-connect`

```
</> apache  
  
AuthType openid-connect
```



Tells Apache:

- use OpenID Connect authentication
-

Understanding `Require valid-user`

```
</> apache  
  
Require valid-user
```



Allows any authenticated Microsoft user.

This is Authentication.

Authentication vs Authorization

Authentication means:

Who is the user?



Authorization means:

What can the user access?



Current setup only performs:

- authentication

Future Authorization Examples

Restrict specific user:

```
</> apache
```



```
Require claim preferred_username:user@example.com
```

Restrict Azure AD group:

```
</> apache
```



```
Require claim groups:GROUP_ID
```

Why This Architecture is Important

Traditional architecture:

```
Application handles authentication
```



Your architecture:

```
Infrastructure handles authentication
```



Benefits:

- centralized security
 - reusable authentication
 - simpler backend apps
 - enterprise scalability
-

Enterprise Use Cases

This architecture is used in:

- API gateways
 - Kubernetes ingress controllers
 - Zero Trust architectures
 - enterprise SSO platforms
 - reverse proxy authentication gateways
-

Summary

Your project successfully implemented:

- Apache reverse proxy
- OpenID Connect authentication
- Microsoft Entra ID integration
- centralized authentication architecture
- protected Azure Web App access
- enterprise-grade authentication flow

This demonstrates how authentication can be externalized from applications into infrastructure using Apache `mod_auth_openidc` and Microsoft Entra ID.